

Mathematical Foundations of AES and Finite-Field-Based Master-Key Preprocessing

Abstract

This report studies several algebraic structures used in AES, beginning with the construction of $\text{GF}(256)$ and polynomial arithmetic over $\text{GF}(2)$. It then explains multiplicative inversion, construction of the AES S-box, AES-128 key expansion, and the standard round transformations. Finally, it presents an exploratory master-key preprocessing construction based on the trace and norm of the quadratic extension $\text{GF}(2^{16})/\text{GF}(2^8)$, a permutation polynomial, and a full-rank linear transformation. The accompanying Python implementation implements the AES procedure represented by the original project code; the additional preprocessing construction is discussed mathematically but is not silently substituted for the standard AES key schedule.

1 Construction of $\text{GF}(256)$

Let L be a field of characteristic 2. For every $a \in L$,

$$a + a = 0.$$

Thus addition has the same XOR behavior as binary addition without carry.

The field $\text{GF}(256)$ can be constructed as

$$K = \text{GF}(2)[x]/(p(x)),$$

where

$$p(x) = x^8 + x^4 + x^3 + x + 1.$$

The polynomial is irreducible over $\text{GF}(2)$, so $(p(x))$ is maximal and K is a field. A basis is

$$\{1, x, x^2, x^3, x^4, x^5, x^6, x^7\}.$$

Consequently,

$$|K| = 2^8 = 256.$$

Each byte corresponds to one polynomial in this basis. Multiplication is ordinary polynomial multiplication over $\text{GF}(2)$ followed by reduction modulo $p(x)$.

2 Polynomial Arithmetic and Multiplicative Inversion

The ring $\text{GF}(2)[x]$ is Euclidean. For $f(x)$ satisfying

$$\gcd(f(x), p(x)) = 1,$$

the extended Euclidean algorithm gives

$$u(x)f(x) + v(x)p(x) = 1.$$

Modulo $p(x)$,

$$u(x)f(x) \equiv 1 \pmod{p(x)},$$

so $u(x)$ represents the inverse of $f(x)$ in K .

The Python implementation in `src/aes.py` represents polynomials as integers whose binary digits are their coefficients. XOR implements addition over $\text{GF}(2)$.

3 The AES S-box

The standard AES S-box is constructed from multiplicative inversion in $\text{GF}(2^8)$ followed by an affine transformation. For an input $a \neq 0$, first compute a^{-1} in $\text{GF}(2^8)$; for $a = 0$, the inverse is defined as 0 for the S-box construction. The affine transformation uses the constant

$$c = 0x63.$$

The implementation generates all 256 S-box values programmatically rather than storing a lookup table.

4 AES-128 Key Expansion

A 16-byte master key is divided into four words. The AES-128 expansion generates 44 words, corresponding to 11 round keys. Every fourth word is processed using `RotWord`, `SubWord`, and the appropriate round constant:

$$w_i = w_{i-4} \oplus \text{Temp},$$

where the transformation of `Temp` includes the S-box and the round constant.

The round constants used by the implementation are

$$01, 02, 04, 08, 10, 20, 40, 80, 1B, 36.$$

5 AES Round Transformations

A 16-byte block is represented as a 4×4 state matrix in column-major order.

5.1 SubBytes

Each state byte is replaced using the AES S-box.

5.2 ShiftRows

Row 0 is unchanged, while rows 1, 2, and 3 are shifted left by 1, 2, and 3 bytes, respectively.

5.3 MixColumns

Each column is multiplied over $\text{GF}(256)$ by

$$M = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix}.$$

The inverse transformation uses

$$M^{-1} = \begin{pmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{pmatrix}.$$

5.4 AddRoundKey

The state and round key are combined bitwise using XOR.

For AES-128, rounds 1 through 9 apply `SubBytes`, `ShiftRows`, `MixColumns`, and `AddRoundKey`. The tenth round omits `MixColumns`.

6 Padding and Encryption Mode

The accompanying implementation applies PKCS#7 padding and encrypts each 16-byte block independently. Thus the implemented mode is ECB.

ECB is retained because it reflects the structure of the original project code. It is not recommended for production use because repeated plaintext blocks produce repeated ciphertext blocks.

7 Proposed Master-Key Preprocessing

The original mathematical proposal explores the extension

$$\text{GF}(2^{16})/\text{GF}(2^8).$$

Let θ satisfy

$$\theta^2 + \theta = 33.$$

Represent a paired pair of $\text{GF}(2^8)$ elements as

$$\beta = a + b\theta.$$

The Frobenius automorphism maps θ to its conjugate $\theta + 1$. Therefore,

$$\begin{aligned} \text{Tr}_{\text{GF}(2^{16})/\text{GF}(2^8)}(\beta) &= \beta + \beta^{2^8} \\ &= (a + b\theta) + (a + b(\theta + 1)) \\ &= b. \end{aligned}$$

Hence

$$\boxed{\text{Tr}_{\text{GF}(2^{16})/\text{GF}(2^8)}(\beta) = b.}$$

Similarly,

$$\begin{aligned} N_{\text{GF}(2^{16})/\text{GF}(2^8)}(\beta) &= \beta\beta^{2^8} \\ &= (a + b\theta)(a + b(\theta + 1)) \\ &= a^2 + ab + b^2(\theta^2 + \theta) \\ &= a^2 + ab + 33b^2. \end{aligned}$$

Thus

$$\boxed{N_{\text{GF}(2^{16})/\text{GF}(2^8)}(\beta) = a^2 + ab + 33b^2.}$$

The proposed nonlinear transformation is

$$v_i = u_i^7 + t_i u_i + n_i + C_i,$$

where

$$t_i = \text{Tr}(u_i), \quad n_i = N(u_i).$$

Since

$$\gcd(7, 2^{16} - 1) = 1,$$

the map

$$u \mapsto u^7$$

is a permutation of $\text{GF}(2^{16})$. A full-rank matrix

$$M \in M_8(\text{GF}(256))$$

can then be used for a linear mixing step

$$V \mapsto MV.$$

This construction is an experimental proposal. It is not part of the standard AES specification, and its cryptographic security would require separate analysis.

8 Implementation and Reproducibility

The Python implementation is organized as follows:

- `poly_degree`: polynomial degree;
- `F2_mult`: polynomial multiplication over $\text{GF}(2)$;
- `F2_divi`: polynomial division with remainder;
- `GF256_mult`: multiplication in $\text{GF}(256)$;
- `GF256_inve`: multiplicative inversion;
- `generate_sbox`: generation of the AES S-box;
- `key_expansion`: AES-128 key expansion;
- `sub_bytes`, `shift_rows`, `mix_columns`, `add_round_key`: AES round transformations;
- `AES_encrypt`: block encryption with PKCS#7 padding;
- `AES_decrypt`: inverse AES transformation.

The example uses

$$K = 000102030405060708090a0b0c0d0e0f$$

and the plaintext “mathematical science”. Running `run_example.py` verifies that encryption followed by decryption recovers the original plaintext.

9 Discussion and Limitations

The project is based on undergraduate algebra, finite-field theory, and the study of AES. The standard AES implementation and the additional mathematical proposal should be distinguished clearly: the former is an implementation of the AES procedure represented by the original code, whereas the latter is an experimental construction for further investigation.

The proposed preprocessing was not fully integrated into the original Python encryption pipeline. Consequently, this repository does not claim that the modified construction is equivalent to, or as secure as, standard AES.

10 AI Assistance

AI assistance was used for some complex implementation details, including polynomial division, multiplicative inversion, and linear transformations. It was also used to identify certain implementation issues. The decryption procedure was developed primarily through function reuse and derivation of the corresponding inverse operations.

References

- [1] L. Nie and S. Ding, *An Introduction to Algebra*, Higher Education Press, Beijing, 2000.
- [2] R. Feng, *Foundations of Finite Fields*, Higher Education Press, Beijing, 2024.